Légende des types de test	
Nominal	Cas passant — comportement attendu nominal (libellé [nominal] dans le fichier)
Alternatif	Cas limite ou variation valide du nominal (libellé [non-nominal] non-erreur)
Erreur	Cas rejetant — entrée invalide, état incohérent ou résultat false attendu
Sécurité	Contrôle de sécurité : falsification de signature ou de payload JWT

TESTS UNITAIRES — Serveur Deno (deno test jwt_test.ts)

createJWT — payload → Promise<string> (4 tests)

ID	Type de test	Nom du test (deno test)	Entrée / Précondition	Appel / Assertions clés	Résultat attendu
JWT-01	Nominal	**createJWT [nominal] — retourne une chaîne à 3 segments base64url**	payload = { id: 1 }	const token = await createJWT({ id: 1 }) token.split('.').length assertMatch(part, /^[A-Za-z0-9\-_]+$/)	3 segments séparés par '.' Chaque segment correspond à du base64url valide [A-Za-z0-9\-_]+
JWT-02	Nominal	**createJWT [nominal] — le header encode bien l'algorithme HS256**	payload = { id: 1 }	const token = await createJWT({ id: 1 }) const header = JSON.parse(atob(token.split('.')[0])) assertEquals(header.alg, 'HS256')	header.alg === "HS256"
JWT-03	Alternatif	**createJWT [non-nominal] — deux appels avec le même payload produisent des tokens différents (nonce/exp)**	payload = { id: 1 } — deux appels séparés par 10 ms	const t1 = await createJWT({ id: 1 }) await sleep(10ms) const t2 = await createJWT({ id: 1 }) assertNotEquals(t1, t2)	t1 !== t2 (exp ou signature diffère entre les deux appels)
JWT-04	Alternatif	**createJWT [non-nominal] — payload sans id → token créé mais verifyJWT retourne null**	payload = {} (cast as { id: number })	const token = await createJWT({} as { id: number }) const payload = await verifyJWT(token) assertEquals(payload, null)	createJWT ne lève pas d'erreur verifyJWT(token) === null (pas de champ id valide)

TESTS UNITAIRES — Serveur Deno (deno test jwt_test.ts)					
verifyJWT — token: string → Promise<AuthPayload | null> (6 tests)					
ID	**Type de test**	**Nom du test (deno test)**	**Entrée / Précondition**	**Appel / Assertions clés**	**Résultat attendu**
VJWT-01	Nominal	**verifyJWT [nominal] — retourne le payload avec id et exp corrects**	const token = await createJWT({ id: 42 })	const payload = await verifyJWT(token) assertEquals(payload?.id, 42) assertEquals(payload!.exp > nowSec && payload!.exp <= in24h + 5, true)	payload.id === 42 nowSec < payload.exp ≤ nowSec + 86 400 s + 5 s (~24 h)
VJWT-02	Erreur	**verifyJWT [non-nominal] — chaîne vide → null**	token = ""	const result = await verifyJWT('') assertEquals(result, null)	result === null
VJWT-03	Erreur	**verifyJWT [non-nominal] — token malformé (2 segments) → null**	token = "abc.def"	const result = await verifyJWT('abc.def') assertEquals(result, null)	result === null (structure JWT invalide : 3 segments requis)
VJWT-04	Sécurité	**verifyJWT [non-nominal] — token avec signature incorrecte → null**	Token valide créé, puis 3e segment remplacé par 'INVALIDE_SIGNATURE_XXXXX'	const parts = token.split('.') const tampered = `${parts[0]}.${parts[1]}.INVALIDE_SIGNATURE_XXXXX` const result = await verifyJWT(tampered) assertEquals(result, null)	result === null (vérification HMAC HS256 échoue sur la signature falsifiée)
VJWT-05	Sécurité	**verifyJWT [non-nominal] — token avec payload altéré (id=999) → null**	Token valide créé pour id=1, puis partie payload remplacée par un payload encodé id=999	const fakePart = btoa(JSON.stringify({ id:999, exp:... })) const tampered = `${parts[0]}.${fakePart}.${parts[2]}` const result = await verifyJWT(tampered) assertEquals(result, null)	result === null (signature ne correspond plus au payload modifié)
VJWT-06	Erreur	**verifyJWT [non-nominal] — token expiré → null**	Token signé avec setExpirationTime('1s') via @panva/jose, puis attente 1 500 ms	const expiredToken = await new SignJWT({ id:1 }).setExpirationTime('1s').sign(JWT_KEY) await sleep(1500ms) const result = await verifyJWT(expiredToken) assertEquals(result, null)	result === null (exp dépassé au moment de la vérification)

TESTS UNITAIRES — Serveur Deno (deno test jwt_test.ts)

hashPassword — password: string → Promise<string> (5 tests)

ID	Type de test	Nom du test (deno test)	Entrée / Précondition	Appel / Assertions clés	Résultat attendu
HASH-01	Nominal	**hashPassword [nominal] — retourne le format hash.salt attendu**	password = "motdepasse"	const stored = await hashPassword('motdepasse') assertMatch(stored, /^[0-9a-f]{128}\.[0-9a-f]{32}$/)	Format : 128 caractères hex + '.' + 32 caractères hex
HASH-02	Nominal	**hashPassword [nominal] — deux hachages du même MDP sont différents (sel aléatoire)**	password = "motdepasse" — deux appels indépendants	const h1 = await hashPassword('motdepasse') const h2 = await hashPassword('motdepasse') assertNotEquals(h1, h2)	h1 !== h2 (sel généré aléatoirement à chaque appel)
HASH-03	Alternatif	**hashPassword [non-nominal] — mot de passe vide → hash valide produit sans erreur**	password = ""	const stored = await hashPassword('') assertMatch(stored, /^[0-9a-f]{128}\.[0-9a-f]{32}$/)	Hash non null, format valide, aucune exception
HASH-04	Alternatif	**hashPassword [non-nominal] — mot de passe très long (1 000 caractères) → aucune exception**	password = 'a'.repeat(1000)	const stored = await hashPassword('a'.repeat(1000)) assertMatch(stored, /^[0-9a-f]{128}\.[0-9a-f]{32}$/)	Hash non null, format valide, aucune exception
HASH-05	Alternatif	**hashPassword [non-nominal] — caractères spéciaux et unicode → aucune exception**	password = "P@$$w0rd! é中文"	const stored = await hashPassword('P@$$w0rd!\u{1F510}\u00E9\u4E2D\u6587') assertMatch(stored, /^[0-9a-f]{128}\.[0-9a-f]{32}$/)	Hash non null, format valide, aucune exception

TESTS UNITAIRES — Serveur Deno (deno test jwt_test.ts)

verifyPassword — (password, storedHash) → Promise<boolean> (6 tests)

ID	Type de test	Nom du test (deno test)	Entrée / Précondition	Appel / Assertions clés	Résultat attendu
VFY-01	Nominal	**verifyPassword [nominal] — MDP correct → true**	const stored = await hashPassword ('secret123')	const result = await verifyPassword('secret123', stored) assertEquals(result, true)	result === true
VFY-02	Nominal	**verifyPassword [nominal] — MDP incorrect → false**	const stored = await hashPassword ('secret123')	const result = await verifyPassword('mauvaismdp', stored) assertEquals(result, false)	result === false
VFY-03	Erreur	**verifyPassword [non-nominal] — MDP vide vs hash d'un MDP non-vide → false**	const stored = await hashPassword ('secret123')	const result = await verifyPassword('', stored) assertEquals(result, false)	result === false
VFY-04	Erreur	**verifyPassword [non-nominal] — hash malformé sans point → exception levée**	storedHash = "hashsanspoint" (aucun '.')	await assertRejects(() => verifyPassword('secret', 'hashsanspoint'), Error)	Exception levée (split('.')[1] → undefined → erreur lors du décodage du sel)

VFY-05	Erreur	**verifyPassword [non-nominal] — hash avec sel invalide (non-hex) → false ou exception**	storedHash = 'a'.repeat(128) + '.' + 'z'.repeat(32)
			(sel non-hex : 'z' invalide)
				try {
				const result = await verifyPassword('secret', fakeHash)
				assertEquals(result, false)
				} catch (_e) {
				// exception aussi acceptée
				}
					result === false
					OU exception levée selon l'implémentation scrypt de la plateforme
					(les deux sont acceptés)

VFY-06	Nominal	**verifyPassword [non-nominal] — même MDP vérifié contre deux hashes différents → true pour les deux**	const h1 = await hashPassword ('motdepasse') const h2 = await hashPassword ('motdepasse') (h1 !== h2 car sels différents)	const r1 = await verifyPassword('motdepasse', h1) const r2 = await verifyPassword('motdepasse', h2) assertEquals(r1, true) assertEquals(r2, true)	r1 === true ET r2 === true (le sel est correctement réutilisé dans chaque vérification)